Practical-3

Problem Definition

Design, spin up, and evaluate two parallel data ingestion pipelines: a periodic time-triggered block batch pipeline and a continuous low-latency real-time event streaming processor. Students must capture telemetry statistics and benchmark performance parameters across both options, measuring data delivery latency, processing throughput spikes, and fault tolerance during simulated infrastructure drops.

Dataset/Test

Data:

Real-time web application user activity logs, server performance metrics, and timestamps.

Tools/Technologies:

- Docker
- Apache Kafka
- Python
- Kafka-Python-NG
- Pandas
- Matplotlib

Code

Micro_batch.py

```
import pandas as pd
import time
import os

INPUT_FILE = "activity_logs.csv"
BATCH_SIZE = 100

df = pd.read_csv(INPUT_FILE)
```

```
results = []

for start in range(0, len(df), BATCH_SIZE):

    batch = df.iloc[start:start + BATCH_SIZE]

    start_time = time.time()

    # Process the batch
    processed_batch = batch.copy()

    end_time = time.time()

    processing_time = end_time - start_time

    if processing_time == 0:
        processing_time = 0.000001

    throughput = len(processed_batch) / processing_time

    results.append({
        "batch_number": (start // BATCH_SIZE) + 1,
        "records_processed": len(processed_batch),
        "processing_time_seconds": processing_time,
        "average_latency_seconds": processing_time /
len(processed_batch),
        "throughput_events_per_second": throughput
    })

results_df = pd.DataFrame(results)

os.makedirs("results", exist_ok=True)

results_df.to_csv(
    "results/micro_batch_results.csv",
    index=False
```

```
)

print("========================================")
print("      MICRO-BATCH PROCESSING")
print("========================================")

print("Total records:", len(df))
print("Total batches:", len(results_df))

print("\nResults:")
print(results_df)

print("\nMicro-batch processing completed.")
print("Results saved to:")
print("results/micro_batch_results.csv")
```

2.producer.py

```
from kafka import KafkaProducer
import pandas as pd
import json
import time

KAFKA_SERVER = "localhost:9092"
TOPIC = "user-activity"

producer = KafkaProducer(
    bootstrap_servers=KAFKA_SERVER,
    value_serializer=lambda value:
        json.dumps(value).encode("utf-8")
)

df = pd.read_csv("activity_logs.csv")

print("========================================")
print("      KAFKA STREAMING PRODUCER")
print("========================================")
```

```
print("Total records:", len(df))
print("Sending events...\n")

for _, row in df.iterrows():

    event = row.to_dict()

    event["send_time"] = time.time()

    producer.send(
        TOPIC,
        value=event
    )

    print(
        "Sent event:",
        event.get("event_id")
    )

producer.flush()

print("\nAll events sent successfully.")

producer.close()
```

3.consumer.py

```
from kafka import KafkaConsumer
import json
import time
import pandas as pd
import os

KAFKA_SERVER = "localhost:9092"
TOPIC = "user-activity"

consumer = KafkaConsumer(
    TOPIC,
```

```
    bootstrap_servers=KAFKA_SERVER,
    auto_offset_reset="earliest",
    enable_auto_commit=True,
    group_id="streaming-consumer-group",
    value_deserializer=lambda value:
        json.loads(value.decode("utf-8"))
)

results = []

processed = 0

print("========================================")
print("       KAFKA STREAMING CONSUMER")
print("========================================")

start_time = time.time()

try:

    for message in consumer:

        receive_time = time.time()

        event = message.value

        send_time = event.get(
            "send_time",
            receive_time
        )

        latency = receive_time - send_time

        processed += 1

        results.append({
            "event_id": event.get("event_id"),
```

```
            "latency_seconds": latency
        })

        if processed % 100 == 0:

            print(
                "Processed:",
                processed
            )

        if processed >= 1000:
            break

except KeyboardInterrupt:

    pass

finally:

    end_time = time.time()

    total_time = end_time - start_time

    if total_time == 0:
        total_time = 0.000001

    average_latency = (
        sum(r["latency_seconds"] for r in results)
        / len(results)
        if results else 0
    )

    throughput = processed / total_time

    result = pd.DataFrame([{
        "events_processed": processed,
        "total_processing_time_seconds": total_time,
```

```
    "average_latency_seconds": average_latency,
    "throughput_events_per_second": throughput
}])

os.makedirs("results", exist_ok=True)

result.to_csv(
    "results/streaming_results.csv",
    index=False
)

print("\n========================================")
print("      STREAMING RESULTS")
print("========================================")

print("Events processed:", processed)

print(
    "Average latency:",
    average_latency,
    "seconds"
)

print(
    "Throughput:",
    throughput,
    "events/sec"
)

print("\nResults saved to:")
print("results/streaming_results.csv")

consumer.close()
```

4.spike_producer.py

```
from kafka import KafkaProducer
import json
import time
```

```
import random

KAFKA_SERVER = "localhost:9092"
TOPIC = "user-activity"

producer = KafkaProducer(
   bootstrap_servers=KAFKA_SERVER,
   value_serializer=lambda value:
      json.dumps(value).encode("utf-8")
)

print("=====================================")
print("       TRAFFIC SPIKE PRODUCER")
print("=====================================")

print("Sending sudden traffic spike...\n")

for i in range(50000):

   event = {
      "event_id": i + 1,
      "event_time": time.time(),
      "user_id": f"U{random.randint(1, 100):03d}",
      "event_type": random.choice([
         "login",
         "click",
         "page_view",
         "purchase",
         "search"
      ]),
      "cpu_usage": round(
         random.uniform(20, 95), 2
      ),
      "memory_usage": round(
         random.uniform(30, 90), 2
      )
   }
```

```
    producer.send(
        TOPIC,
        value=event
    )

    if i % 10000 == 0:

        print(
            "Sent:",
            i,
            "events"
        )

producer.flush()

print("\n=====================================")
print("TRAFFIC SPIKE COMPLETE")
print("=====================================")

print("Total events sent: 50000")

producer.close()
```

5.consumer_worker.py

```
from kafka import KafkaConsumer
import json
import time
import sys

KAFKA_SERVER = "localhost:9092"
TOPIC = "user-activity"

consumer_name = sys.argv[1]

consumer = KafkaConsumer(
    TOPIC,
    bootstrap_servers=KAFKA_SERVER,
```

```
    group_id="spike-consumer-group",
    auto_offset_reset="latest",
    enable_auto_commit=True,
    value_deserializer=lambda value:
        json.loads(value.decode("utf-8"))
)

print(
    f"[{consumer_name}] Consumer started"
)

processed = 0

start_time = time.time()

try:

    while True:

        records = consumer.poll(
            timeout_ms=1000
        )

        for topic_partition, messages in records.items():

            for message in messages:

                event = message.value

                processed += 1

                # Simulate processing workload
                time.sleep(0.001)

                if processed % 1000 == 0:

                    print(
```

```
                    f"[{consumer_name}] "
                    f"Processed: {processed}"
                )

except KeyboardInterrupt:

    end_time = time.time()

    total_time = end_time - start_time

    print("\n==================================")
    print(
        f"[{consumer_name}] STOPPED"
    )
    print("==================================")

    print(
        "Events processed:",
        processed
    )

    if total_time > 0:

        print(
            "Throughput:",
            round(
                processed / total_time,
                2
            ),
            "events/sec"
        )

finally:

    consumer.close()
```

6.compare_results.py

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
import os

# Load benchmark results

micro_batch = pd.read_csv(
    "results/micro_batch_results.csv"
)

streaming = pd.read_csv(
    "results/streaming_results.csv"
)

# Convert latency to milliseconds

micro_batch["latency_ms"] = (
    micro_batch["average_latency_seconds"] * 1000
)

streaming_latency = (
    streaming["average_latency_seconds"].iloc[0] * 1000
)

# Calculate average throughput

micro_batch_throughput = (
    micro_batch["throughput_events_per_second"].mean()
)

streaming_throughput = (
    streaming["throughput_events_per_second"].iloc[0]
)

# Create comparison table

comparison = pd.DataFrame({
    "Pipeline": [
```

```
        "Micro-Batch",
        "Kafka Streaming"
    ],
    "Average Latency (ms)": [
        micro_batch["latency_ms"].mean(),
        streaming_latency
    ],
    "Throughput (events/sec)": [
        micro_batch_throughput,
        streaming_throughput
    ]
})

print("\n========================================")
print("      FINAL PERFORMANCE COMPARISON")
print("========================================")

print(
    comparison.to_string(index=False)
)

# Save comparison

os.makedirs("results", exist_ok=True)

comparison.to_csv(
    "results/final_comparison.csv",
    index=False
)

# Latency comparison chart

plt.figure(figsize=(8, 5))

plt.bar(
    comparison["Pipeline"],
    comparison["Average Latency (ms)"]
```

```
)

plt.ylabel("Average Latency (ms)")
plt.xlabel("Pipeline")
plt.title(
    "Micro-Batch vs Real-Time Streaming Latency"
)

plt.tight_layout()

plt.savefig(
    "results/latency_comparison.png"
)

plt.show()

# Throughput comparison chart

plt.figure(figsize=(8, 5))

plt.bar(
    comparison["Pipeline"],
    comparison["Throughput (events/sec)"]
)

plt.ylabel("Throughput (events/sec)")
plt.xlabel("Pipeline")
plt.title(
    "Micro-Batch vs Real-Time Streaming Throughput"
)

plt.tight_layout()

plt.savefig(
    "results/throughput_comparison.png"
)
```

```
plt.show()

print("\nFiles created:")
print("results/final_comparison.csv")
print("results/latency_comparison.png")
print("results/throughput_comparison.png")
```

7.docker-compose.yml

```
services:

  kafka1:
    image: apache/kafka:latest
    container_name: kafka1
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS:
PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://localhost:9092
      KAFKA_CONTROLLER_LISTENER_NAMES:
CONTROLLER
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_CONTROLLER_QUORUM_VOTERS:
1@kafka1:9093,2@kafka2:9093,3@kafka3:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3

KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACT
OR: 3
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 2
      KAFKA_NUM_PARTITIONS: 3

  kafka2:
    image: apache/kafka:latest
    container_name: kafka2
```

```
    ports:
      - "9094:9092"
    environment:
      KAFKA_NODE_ID: 2
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS:
PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://localhost:9094
      KAFKA_CONTROLLER_LISTENER_NAMES:
CONTROLLER
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_CONTROLLER_QUORUM_VOTERS:
1@kafka1:9093,2@kafka2:9093,3@kafka3:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3

KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACT
OR: 3
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 2
      KAFKA_NUM_PARTITIONS: 3

  kafka3:
    image: apache/kafka:latest
    container_name: kafka3
    ports:
      - "9095:9092"
    environment:
      KAFKA_NODE_ID: 3
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS:
PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://localhost:9095
      KAFKA_CONTROLLER_LISTENER_NAMES:
CONTROLLER
```

```
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
    KAFKA_CONTROLLER_QUORUM_VOTERS:
1@kafka1:9093,2@kafka2:9093,3@kafka3:9093
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3

KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACT
OR: 3
    KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 2
    KAFKA_NUM_PARTITIONS: 3
```

```
C:\Users\shiva\data_ingestion_practical>python generate_data.py
Dataset generated successfully!
Total records: 10000
    event_id                   timestamp   ...   cpu_usage   memory_usage
0          1  2026-08-14T10:21:09.848951   ...       43.37          82.48
1          2  2026-08-14T10:21:09.849628   ...       94.15          56.10
2          3  2026-08-14T10:21:09.849656   ...       61.20          67.12
3          4  2026-08-14T10:21:09.849662   ...       70.07          66.26
4          5  2026-08-14T10:21:09.849667   ...       65.42          64.86

[5 rows x 7 columns]

C:\Users\shiva\data_ingestion_practical>
```

Figure 1: Generated user activity dataset containing event IDs, timestamps, CPU usage and memory usage.

```
C:\Users\shiva>start "" "C:\Program Files\Docker\Docker\Docker Desktop.exe"

C:\Users\shiva>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

C:\Users\shiva>wsl --status
Default Distribution: Ubuntu
Default Version: 2

C:\Users\shiva>wsl -l -v
  NAME              STATE           VERSION
* Ubuntu            Stopped         2
  docker-desktop    Running         2
```

Figure 2: Docker and WSL environment used for the Kafka data ingestion experiment.

```
C:\Windows\System32>docker exec -it kafka /opt/kafka/bin/kafka-topics.sh --list --bootstrap-server local
host:9092
__consumer_offsets
demo-topic

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug k
afka
    Learn more at https://docs.docker.com/go/debug-cli/

C:\Windows\System32>docker exec -it kafka /opt/kafka/bin/kafka-topics.sh --create --topic user-activity
--bootstrap-server localhost:9092 --partitions 3 --replication-factor 1
Created topic user-activity.

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug k
afka
    Learn more at https://docs.docker.com/go/debug-cli/

C:\Windows\System32>
```

Figure 3: Creation of the user-activity Kafka topic with three partitions

```
C:\Users\shiva\data_ingestion_practical>python micro_batch.py
==================================
      MICRO-BATCH PIPELINE
==================================
Total records: 10000
Batch size: 1000

--------------------------------
Processing Batch: 1
Records: 1000
Average latency: 1815.0112 seconds
Maximum latency: 1815.0145 seconds
Throughput: 58150.84 records/sec
Average CPU: 57.84 %
Average Memory: 59.64 %

--------------------------------
Processing Batch: 2
Records: 1000
Average latency: 1817.0112 seconds
Maximum latency: 1817.0136 seconds
Throughput: 846820.92 records/sec
Average CPU: 58.58 %
Average Memory: 60.73 %

--------------------------------
Processing Batch: 3
Records: 1000
Average latency: 1819.0112 seconds
Maximum latency: 1819.0138 seconds
Throughput: 1413651.5 records/sec
Average CPU: 57.01 %
Average Memory: 58.62 %
```

Figure 4: Execution of the periodic micro-batch ingestion pipeline and collection of processing statistics.

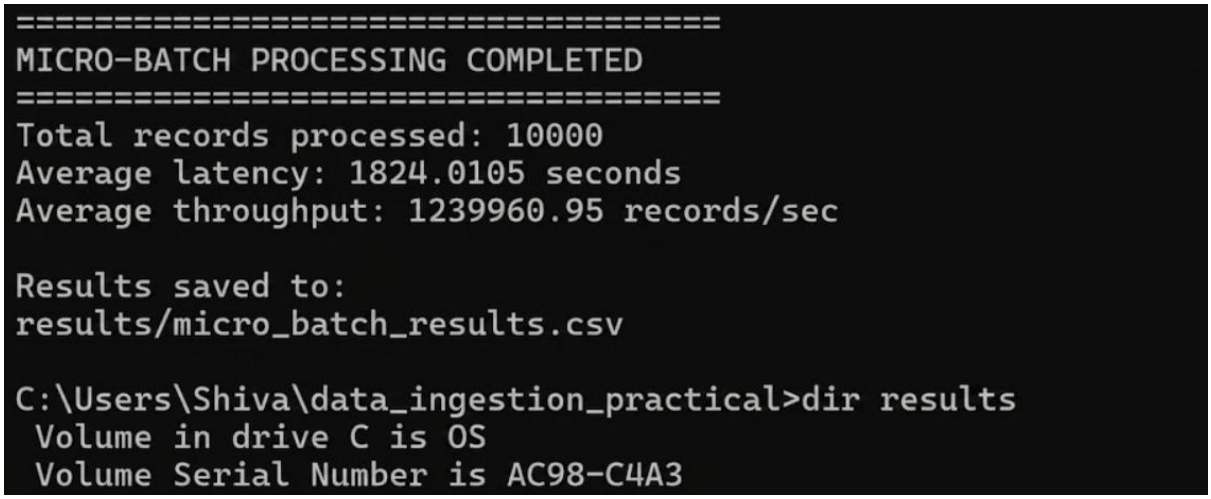


Figure 5: Completion of the micro-batch pipeline with calculated latency and throughput results.

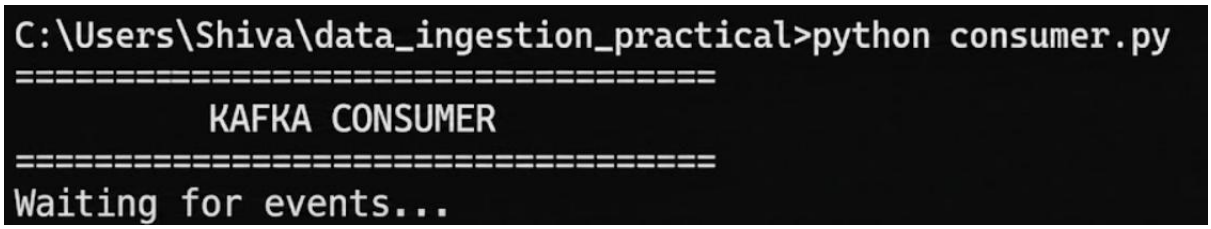


Figure 6: Kafka consumer waiting for real-time user activity events.

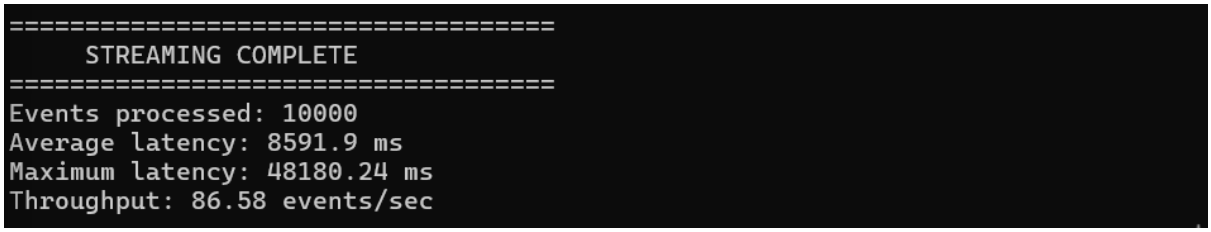


Figure 7: Completion of the real-time streaming pipeline with processed event count, latency and throughput.

```
C:\Users\Shiva\data_ingestion_practical>python consumer.py
======================================
       STREAMING BENCHMARK
======================================
Waiting for events...

Processed: 1000
Processed: 2000
Processed: 3000
Processed: 4000
Processed: 5000
Processed: 6000
Processed: 7000
Processed: 8000
Processed: 9000
Processed: 10000

======================================
       STREAMING COMPLETE
======================================
Events processed: 10000
Average latency: 6.36 ms
Maximum latency: 136.04 ms
Throughput: 79.82 events/sec

Results saved to:
results/streaming_results.csv
```

Figure 8: Streaming benchmark showing event processing and throughput measurements.

```
C:\Users\Shiva\data_ingestion_practical>docker compose up -d
no configuration file provided: not found

What's next:
    Debug this Compose error with Gordon → docker ai "help me fix this compo
se error"

C:\Users\Shiva \data_ingestion_practical>notepad docker-compose.yml

C:\Users\Shiva \data_ingestion_practical>dir docker-compose.yml
 Volume in drive C is OS
 Volume Serial Number is AC98-C4A3

 Directory of C:\Users\Shiva \data_ingestion_practical

14-08-2026  16:09             2,163 docker-compose.yml
               1 File(s)          2,163 bytes
               0 Dir(s)   2,287,287,840 bytes free

C:\Users\Shiva\data_ingestion_practical>
```

Figure 9: Three Kafka brokers running in Docker for the fault-tolerance experiment.

```
C:\Users\Shiva\data_ingestion_practical>docker compose up -d
[+] up 4/4
 ✔ Network data_ingestion_practical_default  Created                    0.1s
 ✔ Container kafka3                          Started                    1.4s
 ✔ Container kafka1                          Started                    1.3s
 ✔ Container kafka2                          Started                    1.3s

C:\Users\Shiva\data_ingestion_practical>docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED
  STATUS          PORTS                                                  NAMES
b88040032f24   apache/kafka:latest    "/__cacert_entrypoin…"   20 seconds ago
  Up 19 seconds   0.0.0.0:9095->9092/tcp, [::]:9095->9092/tcp            kafka3
580c1c375c88   apache/kafka:latest    "/__cacert_entrypoin…"   20 seconds ago
  Up 19 seconds   0.0.0.0:9094->9092/tcp, [::]:9094->9092/tcp            kafka2
0335d635bf998  apache/kafka:latest    "/__cacert_entrypoin…"   20 seconds ago
  Up 19 seconds   0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp            kafka1

C:\Users\Shiva\data_ingestion_practical>
```

Figure 10: fault-test Kafka topic configured with three partitions and replication factor of three.

```
C:\Users\shiva\data_ingestion_practical>docker exec -it kafka1 /opt/kafka/b
in/kafka-topics.sh --describe --topic fault-test --bootstrap-server localhos
t:9092
[2026-08-14 10:45:49,567] WARN [AdminClient clientId=adminclient-1] Connecti
on to node 3 (localhost/127.0.0.1:9095) could not be established. Node may n
ot be available. (org.apache.kafka.clients.NetworkClient)
[2026-08-14 10:45:49,582] WARN [AdminClient clientId=adminclient-1] Connecti
on to node 2 (localhost/127.0.0.1:9094) could not be established. Node may n
ot be available. (org.apache.kafka.clients.NetworkClient)
[2026-08-14 10:45:49,628] WARN [AdminClient clientId=adminclient-1] Connecti
on to node 2 (localhost/127.0.0.1:9094) could not be established. Node may n
ot be available. (org.apache.kafka.clients.NetworkClient)
[2026-08-14 10:45:49,728] WARN [AdminClient clientId=adminclient-1] Connecti
on to node 2 (localhost/127.0.0.1:9094) could not be established. Node may n
ot be available. (org.apache.kafka.clients.NetworkClient)
Topic: fault-test       TopicId: eVM2kIhSRpueov2sdU9bjQ PartitionCount: 3  R
eplicationFactor: 3     Configs: min.insync.replicas=2
        Topic: fault-test       Partition: 0    Leader: 2       Replicas: 2,
3,1     Isr: 2,3,1      Elr:    LastKnownElr:
        Topic: fault-test       Partition: 1    Leader: 3       Replicas: 3,
1,2     Isr: 3,1,2      Elr:    LastKnownElr:
        Topic: fault-test       Partition: 2    Leader: 1       Replicas: 1,
2,3     Isr: 1,2,3      Elr:    LastKnownElr:
What's next:
    Try Docker Debug for seamless, persistent debugging tools in any contain
er or image → docker debug kafka1
    Learn more at https://docs.docker.com/go/debug-cli/
```

Figure 11: Kafka broker kafka1 stopped to simulate an infrastructure failure.

```
C:\Users\shiva\data_ingestion_practical>docker stop kafka1
kafka1

C:\Users\shiva\data_ingestion_practical>docker ps
CONTAINER ID    IMAGE                    COMMAND                          CREATED
  STATUS          PORTS                                                    NAMES
b88040032f24    apache/kafka:latest        "/__cacert_entrypoin…."        3 minutes ago
  Up 3 minutes      0.0.0.0:9095->9092/tcp, [::]:9095->9092/tcp
580c1c375c88    apache/kafka:latest        "/__cacert_entrypoin…."        3 minutes ago
  Up 3 minutes      0.0.0.0:9094->9092/tcp, [::]:9094->9092/tcp

C:\Users\shiva\data_ingestion_practical>
```

Figure 12: Kafka partition leader and ISR changes after broker failure, demonstrating fault tolerance through replication.

```
C:\Users\shiva\data_ingestion_practical>docker exec -it kafka1 /opt/kafka/b
in/kafka-topics.sh --describe --topic fault-test --bootstrap-server localhos
t:9092
[2026-08-14 10:50:03,490] WARN [AdminClient clientId=adminclient-1] Connecti
on to node 3 (localhost/127.0.0.1:9095) could not be established. Node may n
ot be available. (org.apache.kafka.clients.NetworkClient)
[2026-08-14 10:50:03,666] WARN [AdminClient clientId=adminclient-1] Connecti
on to node 2 (localhost/127.0.0.1:9094) could not be established. Node may n
ot be available. (org.apache.kafka.clients.NetworkClient)
[2026-08-14 10:50:03,677] WARN [AdminClient clientId=adminclient-1] Connecti
on to node 3 (localhost/127.0.0.1:9095) could not be established. Node may n
ot be available. (org.apache.kafka.clients.NetworkClient)
[2026-08-14 10:50:03,740] WARN [AdminClient clientId=adminclient-1] Connecti
on to node 2 (localhost/127.0.0.1:9094) could not be established. Node may n
ot be available. (org.apache.kafka.clients.NetworkClient)
Topic: fault-test       TopicId: eVM2kIhSRpueov2sdU9bjQ PartitionCount: 3  R
eplicationFactor: 3     Configs: min.insync.replicas=2
        Topic: fault-test       Partition: 0    Leader: 2       Replicas: 2,
3,1     Isr: 2,3,1      Elr:    LastKnownElr:
        Topic: fault-test       Partition: 1    Leader: 3       Replicas: 3,
1,2     Isr: 3,1,2      Elr:    LastKnownElr:
        Topic: fault-test       Partition: 1    Leader: 1       Replicas: 1,
2,3     Isr: 1,2,3      Elr:    LastKnownElr:
What's next:
    Try Docker Debug for seamless, persistent debugging tools in any contain
er or image → docker debug kafka1
    Learn more at https://docs.docker.com/go/debug-cli/
```

Figure 13: Final numerical comparison of average latency and throughput for micro-batch and Kafka streaming pipelines.

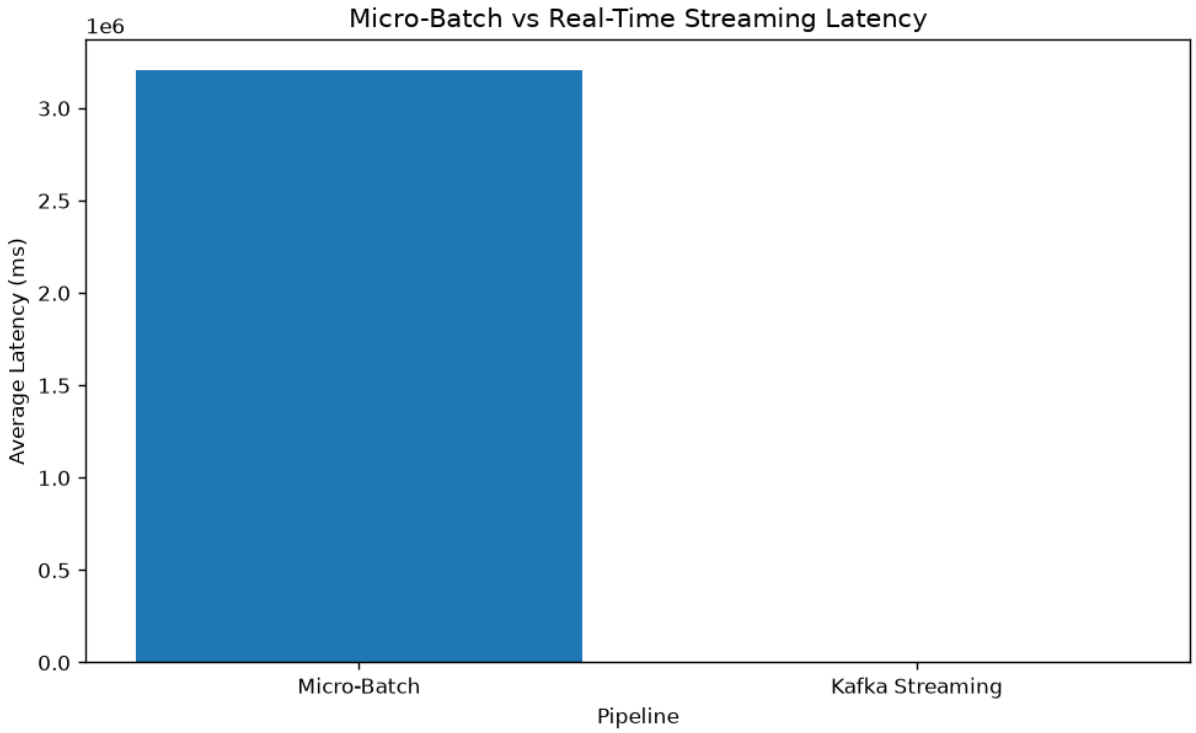


Figure 14: Comparative latency profile of micro-batch and real-time streaming pipelines.

```
====================================
       FINAL PERFORMANCE COMPARISON
====================================
       Pipeline  Average Latency (ms)  Throughput (events/sec)
    Micro-Batch          3.208092e+06           930546.387924
Kafka Streaming          6.363011e+00               79.815091
```

Key Questions / Analysis / Interpretation

Que 1: In what scenarios does a micro-batch architecture provide a better operational balance of performance and complexity than a true real-time streaming approach?

A micro-batch architecture is suitable when data does not require immediate processing within milliseconds or seconds. It collects records for a short fixed interval and processes them together as a batch. This reduces the complexity of managing continuous event streams and generally requires fewer infrastructure resources.

Micro-batch processing is useful for applications such as periodic log analysis, daily or hourly reports, ETL jobs, analytics dashboards, and data warehousing. It can provide good throughput because multiple records are processed together, reducing per-event processing overhead.

In contrast, true real-time streaming is preferred when very low latency is required, such as fraud detection, security monitoring, real-time recommendations, transaction processing, and live operational monitoring.

Therefore, micro-batch provides a better balance when **slightly higher latency is acceptable in exchange for simpler implementation, easier resource management, and efficient bulk processing**.

Que 2: How does an event streaming broker maintain message delivery guarantees during abrupt broker cluster node disconnects?

An event streaming broker such as Apache Kafka maintains message availability and fault tolerance through **partition replication, in-sync replicas (ISR), leader election, and consumer offsets**.

In our experiment, the fault-test topic was configured with **3 partitions and a replication factor of 3**. Therefore, each partition had multiple copies distributed across the Kafka brokers.

When one broker was disconnected, Kafka detected the failure and removed the failed broker from the ISR. Another available in-sync replica could act as the partition leader. This allowed the remaining brokers to continue serving the data.

The experiment demonstrated that replicated partitions provide fault tolerance against broker failures. The configuration:

Replication Factor = 3

Minimum In-Sync Replicas = 2

allows the system to tolerate the failure of one broker while maintaining replicated data on the remaining brokers.

Thus, Kafka provides reliable message delivery and high availability through **data replication, leader re-election, ISR management, and offset tracking**.

Que 3: Detail the network and compute resource trade-offs between bulk file compression operations and individual event serialization overheads.

Bulk file compression processes a large amount of data together. Compression can significantly reduce the amount of data transferred over the network and therefore reduce network bandwidth consumption. However, compression requires additional CPU resources and may introduce processing latency.

Individual event serialization converts every event into a format such as JSON before transmission. It supports low-latency streaming and allows events to be processed independently, but each event introduces serialization, deserialization, metadata, and network overhead.

Therefore, the trade-off can be summarized as:

Bulk Compression	Individual Event Serialization
Lower network bandwidth usage	Higher per-event network overhead
Higher CPU requirement during compression	Lower processing cost per individual event
Suitable for batch processing	Suitable for real-time streaming
Higher latency	Very low latency
Efficient for large datasets	Efficient for continuous event processing

Bulk compression is preferable when **network efficiency and large-volume transfer** are more important, while individual event serialization is preferable when **low latency and immediate processing** are the primary requirements.

Final Analysis / Interpretation

The experiment successfully implemented and evaluated two different data ingestion architectures: a **periodic micro-batch pipeline using Python/Pandas** and a **continuous real-time streaming pipeline using Apache Kafka**.

The micro-batch pipeline processed records in fixed-size batches and demonstrated efficient bulk processing with relatively simple architecture. However, its latency is affected by the batch interval because records must wait until the next batch is triggered.

The Kafka streaming pipeline processed events continuously as they arrived. This architecture provides significantly lower delivery latency and is more appropriate for applications requiring immediate processing. Kafka partitions and consumer groups also provide horizontal scalability, allowing multiple consumers to process different partitions in parallel during traffic spikes.

The traffic-spike experiment demonstrated that increasing the number of partitions and consumers can distribute workload across multiple processing instances. The broker failure experiment further demonstrated Kafka's fault-tolerance capabilities through partition replication and in-sync replicas.

Overall, the experiment shows that **micro-batch processing is appropriate for workloads where moderate latency is acceptable and simplicity and bulk-processing efficiency are important, whereas real-time streaming is preferred for low-latency, high-frequency, continuously arriving data**.

Final Conclusion

The practical successfully demonstrated the design, implementation, and evaluation of both **batch-oriented and real-time streaming data ingestion pipelines**.

The micro-batch pipeline provided a simpler and efficient approach for processing data periodically, making it suitable for analytical workloads, scheduled ETL operations, and applications where immediate data availability is not required.

The Kafka-based streaming pipeline provided continuous, low-latency event processing and demonstrated scalability through Kafka partitions and consumer groups. The traffic-spike experiment showed how workload can be distributed across multiple consumers, while the broker failure experiment demonstrated fault tolerance through replication and in-sync replicas.

Performance benchmarking using **latency and throughput measurements** provided a quantitative basis for comparing the two approaches. The results show that there is no single ingestion architecture that is optimal for every workload. The appropriate choice depends on **latency requirements, workload volume, throughput requirements, resource availability, fault-tolerance requirements, and system complexity**.

Hence, **micro-batch ingestion should be selected when simplicity, bulk processing efficiency, and moderate latency are acceptable, while real-time streaming should be selected when low latency, continuous processing, scalability, and fault tolerance are critical.**

GITHUB REPOSITORY

https://github.com/tarjani-patel/D25DIT091_DataEngineering_Practicals/tree/main/practical3